



LABORATORIO + TAREA 3

Comunicación Segura con ESP-NOW

PARTE I — Laboratorio Base (Todos los grupos)

Esta primera parte la realizan TODOS los grupos en la misma clase. El objetivo es construir una base común: un sistema de comunicación ESP-NOW con cifrado XOR funcional, que cada grupo luego reemplazará por su mecanismo asignado.

Materiales (por grupo)

- 2 placas ESP32 (cualquier variante: DevKit, WROOM, WROVER)
- 2 cables USB para programación
- Arduino IDE 2.x o PlatformIO instalado y configurado
- ESP32 Arduino Core instalado (Boards Manager → esp32 by Espressif)

Paso 0 — Obtener la dirección MAC de cada ESP32

Sube este sketch a cada placa y anota la dirección MAC mostrada en el Monitor Serial:

```
#include <WiFi.h>

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  Serial.print("MAC: ");
  Serial.println(WiFi.macAddress());
}
void loop() {}
```

Dispositivo	Dirección MAC (anota aquí)
ESP32-A (Transmisor)	_____
ESP32-B (Receptor)	_____

Paso 1 — Comunicación ESP-NOW sin cifrado (texto plano)

Primero verificamos que ESP-NOW funciona antes de añadir cifrado. Observa que el mensaje viaja en texto legible — cualquier dispositivo ESP32 cercano podría capturarlo.

Código ESP32-A — Transmisor

```
#include <esp_now.h>
#include <WiFi.h>

// ⚠ Reemplaza con la MAC real de tu ESP32-B
uint8_t macReceptor[] = {0xFF,0xFF,0xFF,0xFF,0xFF,0xFF};

typedef struct { char texto[200]; uint32_t contador; } Paquete;

void onEnviado(const uint8_t *mac, esp_now_send_status_t estado) {
```

```

    Serial.println(estado == ESP_NOW_SEND_SUCCESS ? "OK" : "FALLO");
}

void setup() {
    Serial.begin(115200);
    WiFi.mode(WIFI_STA);
    esp_now_init();
    esp_now_register_send_cb(onEnviado);
    esp_now_peer_info_t peer = {};
    memcpy(peer.peer_addr, macReceptor, 6);
    peer.channel = 0; peer.encrypt = false;
    esp_now_add_peer(&peer);
}

void loop() {
    static uint32_t cnt = 0;
    Paquete pkt;
    sprintf(pkt.texto, sizeof(pkt.texto), "Hola desde ESP32-A!");
    pkt.contador = cnt++;
    esp_now_send(macReceptor, (uint8_t*)&pkt, sizeof(pkt));
    delay(2000);
}

```

Código ESP32-B — Receptor

```

#include <esp_now.h>
#include <WiFi.h>

typedef struct { char texto[200]; uint32_t contador; } Paquete;

void onRecibido(const uint8_t *mac, const uint8_t *datos, int len) {
    Paquete *pkt = (Paquete*)datos;
    Serial.printf("[#%u] Recibido: %s\n", pkt->contador, pkt->texto);
}

void setup() {
    Serial.begin(115200);
    WiFi.mode(WIFI_STA);
    esp_now_init();
    esp_now_register_rcv_cb(onRecibido);
}

void loop() {}

```

☒ Verificación Paso 1

El Monitor Serial del ESP32-B muestra los mensajes enviados por A.

Pregunta: ¿qué información sensible NO deberías enviar así? ¿Por qué?

Paso 2 — Cifrado XOR con clave compartida

El cifrado XOR aplica la operación bit a bit ($\oplus$) entre cada byte del mensaje y la clave. Es simple, reversible con la misma operación, y sirve como punto de partida para entender el cifrado de flujo.

Fórmula: $\text{byte_cifrado} = \text{byte_mensaje} \oplus \text{byte_clave}$ | $\text{byte_mensaje} = \text{byte_cifrado} \oplus \text{byte_clave}$

```
#include <esp_now.h>
#include <WiFi.h>
#include <string.h>

// — Clave compartida (ambos ESP32 deben tener la misma) —
const uint8_t CLAVE[16] = {
    0x4C, 0xAB, 0x12, 0xF3, 0x9E, 0x71, 0x3D, 0x08,
    0xCC, 0x55, 0xA2, 0x6B, 0x1F, 0xE9, 0x84, 0x37
};

// — Estructura de paquete seguro —
typedef struct __attribute__((packed)) {
    uint8_t  payload[200]; // datos cifrados
    uint8_t  longitud;     // longitud efectiva
    uint32_t id;           // identificador de mensaje
} PaqueteSeguro;

// — Cifrar / Descifrar (XOR es simétrico) —
void xorCifrar(const uint8_t *entrada, uint8_t *salida,
               size_t lon, const uint8_t *clave, size_t lonClave) {
    for (size_t i = 0; i < lon; i++)
        salida[i] = entrada[i] ^ clave[i % lonClave];
}

// — En el transmisor: cifrar antes de enviar —
void enviarSeguro(const char *mensaje) {
    PaqueteSeguro pkt;
    pkt.longitud = strlen(mensaje);
    pkt.id = millis();
    xorCifrar((uint8_t*)mensaje, pkt.payload,
              pkt.longitud, CLAVE, sizeof(CLAVE));
    esp_now_send(macReceptor, (uint8_t*)&pkt, sizeof(pkt));
    Serial.printf("[TX] Enviado cifrado: %d bytes\n", pkt.longitud);
}

// — En el receptor: descifrar al recibir —
void onRecibido(const uint8_t *mac, const uint8_t *datos, int len) {
    PaqueteSeguro *pkt = (PaqueteSeguro*)datos;
    uint8_t claro[201] = {0};
    xorCifrar(pkt->payload, claro, pkt->longitud,
              CLAVE, sizeof(CLAVE));
    Serial.printf("[RX] #%u] %s\n", pkt->id, claro);
}
```

☒ Verificación Paso 2

Escribe un mensaje en el Monitor Serial de A y presiona Enter → B lo muestra descifrado.

Abre el Monitor Serial de A: los bytes enviados son ilegibles (cifrados).

Modifica 1 byte de la CLAVE en el receptor — el mensaje aparece como basura.

Paso 3 — Experimento: limitaciones del XOR

Antes de que cada grupo trabaje en su mecanismo asignado, comprueba estas vulnerabilidades del XOR en clase:

1. Known-plaintext attack: si el atacante conoce un mensaje y su versión cifrada, obtiene la clave con un solo XOR. Pruébalo: $\text{clave} = \text{cifrado} \oplus \text{texto_conocido}$.
2. Reutilización de clave: envía dos mensajes distintos cifrados con la misma clave. Haz XOR de ambos cifrados — la clave desaparece y se filtra información del texto.
3. Clave corta: con CLAVE de 1 byte, el cifrado se rompe con análisis de frecuencia en segundos.

Reflexión final de clase

El XOR es correcto matemáticamente, pero inseguro en la práctica.

La clave debe ser: aleatoria, del mismo largo que el mensaje, y usarse UNA SOLA VEZ (→ One-Time Pad).

Como eso es impráctico, existen AES, ChaCha20 y los demás mecanismos que investigará cada grupo.

La tarea de cada grupo es responder: ¿cómo resuelve MI mecanismo estos problemas del XOR?

Preguntas (todos los grupos)

4. ¿Cuántas claves diferentes de 16 bytes existen? Expresa el número y compáralo con las claves de César (255 posibilidades).
5. Demuestra algebraicamente por qué $(M \oplus K) \oplus K = M$. ¿Qué propiedad del XOR hace esto posible?
6. Si capturas dos mensajes cifrados $C1 = M1 \oplus K$ y $C2 = M2 \oplus K$ con la misma clave, ¿qué obtienes al calcular $C1 \oplus C2$? ¿Qué revela eso?
7. ¿Qué debería cambiar en el sistema XOR para que fuera tan seguro como AES? (Responde en términos de aleatoridad y longitud de clave)

PARTE II — Tarea por Grupo (Mecanismo Asignado)

Una vez completado el laboratorio base, cada grupo profundiza en el mecanismo de seguridad asignado. El código del laboratorio (estructura ESP-NOW + paquete + send/receive) es la base — solo deben reemplazar la función `xorCifrar()` por su mecanismo.

Entregables comunes a todos los grupos

1. Código Arduino funcional con el mecanismo implementado (reemplaza `xorCifrar`)
2. Slides (máx. 8) cubriendo los 6 criterios de exposición
3. Demo en vivo: mostrar mensaje cifrado y descifrado funcionando entre dos ESP32
4. Análisis comparativo: ¿cómo resuelve tu mecanismo las 3 vulnerabilidades del XOR?
5. Informe PDF respondiendo las preguntas específicas de tu grupo

GRUPO 1 · Cifrado César sobre ESP-NOW

Protocolo: Cifrado César (desplazamiento de caracteres)

El cifrado más antiguo del mundo, adaptado a comunicación inalámbrica

Objetivo

Reemplazar el cifrado XOR del laboratorio base por el cifrado César, donde cada byte del mensaje se desplaza un valor fijo (la clave). Analizar sus fortalezas y debilidades.

Conceptos clave a dominar

- Cifrado por sustitución: cada carácter se reemplaza por otro a distancia fija
- Clave: un número del 0 al 255 (desplazamiento)
- Cifrado: $\text{byte_cifrado} = (\text{byte_original} + \text{clave}) \% 256$
- Descifrado: $\text{byte_original} = (\text{byte_cifrado} - \text{clave} + 256) \% 256$
- Ataque de fuerza bruta: solo 256 claves posibles — se rompe en segundos

Tareas de implementación

8. Implementar `cifrarCesar(uint8_t *datos, size_t lon, uint8_t clave)` en el ESP32
9. Enviar un mensaje cifrado por ESP-NOW y verificar que el receptor lo descifra correctamente
10. Implementar un ataque de fuerza bruta en Python que pruebe las 255 claves posibles
11. Medir cuántos milisegundos tarda el ataque — comparar con XOR

Punto de partida — integración con el lab base

En el código del laboratorio, localiza la función `xorCifrar()` y reemplázala por la llamada a tu mecanismo. La estructura del paquete y la lógica de ESP-NOW NO cambian:

```
// — ANTES (laboratorio base) —
void enviarSeguro(const char *mensaje) {
    PaqueteSeguro pkt;
    pkt.longitud = strlen(mensaje);
    // TODO Grupo 1: reemplaza esta línea por tu mecanismo
    xorCifrar((uint8_t*)mensaje, pkt.payload, pkt.longitud, CLAVE, sizeof(CLAVE));
    esp_now_send(macReceptor, (uint8_t*)&pkt, sizeof(pkt));
}

// — DESPUÉS (tu implementación) —
void enviarSeguro(const char *mensaje) {
    PaqueteSeguro pkt;
    pkt.longitud = strlen(mensaje);
    tuMecanismo_cifrar((uint8_t*)mensaje, pkt.payload, pkt.longitud /*, params */);
    esp_now_send(macReceptor, (uint8_t*)&pkt, sizeof(pkt));
}
```

Preguntas que debes responder en la exposición

- ¿Por qué el César es vulnerable con solo ver la frecuencia de los bytes más comunes?
- ¿Aumentar la clave a 2 bytes (65.535 combinaciones) lo haría seguro? ¿Por qué no?
- ¿Qué diferencia hay entre seguridad por oscuridad y seguridad real?

Análisis comparativo obligatorio (incluir en slides)

Compara tu mecanismo con el XOR del laboratorio en estos 3 ejes:

1. Known-plaintext attack: ¿es vulnerable? ¿Por qué sí o no?
2. Reutilización de clave: ¿qué pasa si usas la misma clave dos veces?
3. Fuerza bruta: ¿cuántas claves posibles tiene tu mecanismo? ¿Cuánto tiempo tardaría un ataque?

Recursos sugeridos

- [Cryptoclub.org](https://cryptoclub.org) — Historia y matemática del cifrado César
- dcode.fr/caesar-cipher — Herramienta online de cifrado y ataque
- Script Python de fuerza bruta: iterar clave de 0 a 255 e imprimir resultado

Entregable específico de este grupo:

Código Arduino funcional + script de ataque Python + slides con demo del ataque en vivo

GRUPO 2 · Cifrado Vigenère sobre ESP-NOW

Protocolo: Cifrado Vigenère (clave polialfabética)

La mejora del César: una clave que cambia con cada byte

Objetivo

Implementar el cifrado de Vigenère, donde la clave es una cadena de bytes que se repite cíclicamente. Cada byte del mensaje usa un desplazamiento diferente según la posición.

Conceptos clave a dominar

- Clave polialfabética: la clave se repite: CLAVE CLAVE CLAVE...
- Cifrado: $\text{byte_cifrado} = (\text{byte_original} + \text{clave}[i \% \text{len_clave}]) \% 256$
- Descifrado: $\text{byte_original} = (\text{byte_cifrado} - \text{clave}[i \% \text{len_clave}] + 256) \% 256$
- Índice de coincidencia: herramienta para detectar el largo de la clave
- Ataque de Kasiski: encuentra repeticiones para deducir el largo de clave

Tareas de implementación

12. Implementar `cifrarVigenere(uint8_t *datos, size_t lon, uint8_t *clave, size_t lonClave)`
13. Probar con claves de 1, 4 y 16 bytes — observar cómo cambia la resistencia al ataque
14. Implementar el índice de coincidencia en Python para estimar el largo de la clave
15. Comparar la resistencia de Vigenère vs César con el mismo texto

Punto de partida — integración con el lab base

En el código del laboratorio, localiza la función `xorCifrar()` y reemplázala por la llamada a tu mecanismo. La estructura del paquete y la lógica de ESP-NOW NO cambian:

```
// — ANTES (laboratorio base) —
void enviarSeguro(const char *mensaje) {
    PaqueteSeguro pkt;
    pkt.longitud = strlen(mensaje);
    // TODO Grupo 2: reemplaza esta línea por tu mecanismo
    xorCifrar((uint8_t*)mensaje, pkt.payload, pkt.longitud, CLAVE, sizeof(CLAVE));
    esp_now_send(macReceptor, (uint8_t*)&pkt, sizeof(pkt));
}

// — DESPUÉS (tu implementación) —
void enviarSeguro(const char *mensaje) {
    PaqueteSeguro pkt;
    pkt.longitud = strlen(mensaje);
    tuMecanismo_cifrar((uint8_t*)mensaje, pkt.payload, pkt.longitud /*, params */);
    esp_now_send(macReceptor, (uint8_t*)&pkt, sizeof(pkt));
}
```

Preguntas que debes responder en la exposición

- Si la clave tiene el mismo largo que el mensaje y es aleatoria, ¿qué cifrado obtenemos?
- ¿Por qué repetir la clave es la debilidad fundamental de Vigenère?
- ¿Cómo se relaciona esto con el concepto de One-Time Pad?

Análisis comparativo obligatorio (incluir en slides)

Compara tu mecanismo con el XOR del laboratorio en estos 3 ejes:

1. Known-plaintext attack: ¿es vulnerable? ¿Por qué sí o no?
2. Reutilización de clave: ¿qué pasa si usas la misma clave dos veces?
3. Fuerza bruta: ¿cuántas claves posibles tiene tu mecanismo? ¿Cuánto tiempo tardaría un ataque?

Recursos sugeridos

- dcode.fr/vigenere-cipher — Cifrador y atacante online
- "The Index of Coincidence" — Friedman, 1922 (artículo histórico)
- Python: implementar índice de coincidencia en 20 líneas

Entregable específico de este grupo:

Código Arduino + análisis de índice de coincidencia + demo comparando claves de distinto largo

GRUPO 3 · One-Time Pad sobre ESP-NOW

Protocolo: One-Time Pad (OTP) — cifrado teóricamente perfecto

El único cifrado matemáticamente imposible de romper — y por qué no se usa

Objetivo

Implementar el One-Time Pad: una clave completamente aleatoria del mismo largo que el mensaje, usada una sola vez. Demostrar por qué es perfectamente seguro en teoría pero impráctico en la realidad.

Conceptos clave a dominar

- OTP: clave aleatoria de exactamente el mismo largo que el mensaje
- Cifrado: $\text{byte_cifrado} = \text{byte_original} \text{ XOR } \text{clave_aleatoria}[i]$
- Seguridad perfecta (Shannon, 1949): un texto cifrado es compatible con cualquier mensaje
- El hardware RNG del ESP32: `esp_random()` genera bytes verdaderamente aleatorios
- Problema práctico: ¿cómo compartir la clave de forma segura si es tan larga como el mensaje?

Tareas de implementación

16. Generar una clave aleatoria con `esp_random()` del mismo largo que el mensaje
17. Implementar cifrado/descifrado OTP y verificar que funciona en ESP-NOW
18. Demostrar que reusar la clave (Two-Time Pad) rompe la seguridad: cifrar dos mensajes distintos con la misma clave y XOR los cifrados — observar la filtración
19. Calcular cuántos bytes de clave necesitaría un chat de 1 hora para ser OTP puro

Punto de partida — integración con el lab base

En el código del laboratorio, localiza la función `xorCifrar()` y reemplázala por la llamada a tu mecanismo. La estructura del paquete y la lógica de ESP-NOW NO cambian:

```
// — ANTES (laboratorio base) —
void enviarSeguro(const char *mensaje) {
    PaqueteSeguro pkt;
    pkt.longitud = strlen(mensaje);
    // TODO Grupo 3: reemplaza esta línea por tu mecanismo
    xorCifrar((uint8_t*)mensaje, pkt.payload, pkt.longitud, CLAVE, sizeof(CLAVE));
    esp_now_send(macReceptor, (uint8_t*)&pkt, sizeof(pkt));
}

// — DESPUÉS (tu implementación) —
void enviarSeguro(const char *mensaje) {
    PaqueteSeguro pkt;
    pkt.longitud = strlen(mensaje);
    tuMecanismo_cifrar((uint8_t*)mensaje, pkt.payload, pkt.longitud /*, params */);
    esp_now_send(macReceptor, (uint8_t*)&pkt, sizeof(pkt));
}
```

Preguntas que debes responder en la exposición

- ¿Por qué Shannon demostró que el OTP es perfectamente seguro?
- ¿Qué pasa exactamente si se reutiliza la clave? Muestra el ataque con números.

- ¿Cómo resuelve AES el problema de distribución de claves que tiene el OTP?

Análisis comparativo obligatorio (incluir en slides)

Compara tu mecanismo con el XOR del laboratorio en estos 3 ejes:

1. Known-plaintext attack: ¿es vulnerable? ¿Por qué sí o no?
2. Reutilización de clave: ¿qué pasa si usas la misma clave dos veces?
3. Fuerza bruta: ¿cuántas claves posibles tiene tu mecanismo? ¿Cuánto tiempo tardaría un ataque?

Recursos sugeridos

- Shannon, C.E. (1949) — Communication Theory of Secrecy Systems (paper original)
- crypto.stackexchange.com — "Why is OTP perfect secrecy?"
- Documentación ESP32: `esp_random()` y el hardware RNG

Entregable específico de este grupo:

Código Arduino con OTP + demo del ataque Two-Time Pad + cálculo de viabilidad práctica

GRUPO 4 · AES-128 sobre ESP-NOW

Protocolo: AES-128 en modo CTR — el cifrado simétrico estándar

Del cifrado casero al estándar mundial — usando mbedTLS integrado en el ESP32

Objetivo

Reemplazar el cifrado XOR por AES-128 en modo CTR usando la biblioteca mbedTLS que viene integrada en el ESP32. Comprender por qué AES es el sucesor de todos los cifrados anteriores.

Conceptos clave a dominar

- AES: cifrado de bloque de 128 bits, claves de 128 o 256 bits
- Modo CTR: convierte AES de bloque a flujo — ideal para mensajes de largo variable
- Nonce/IV: número de uso único que garantiza que dos mensajes iguales cifren diferente
- mbedTLS: biblioteca criptográfica incluida en el ESP32 Arduino Core
- Por qué AES venció a DES, 3DES y RC4 (concurso NIST 1997–2001)

Tareas de implementación

20. Implementar cifrado/descifrado con `mbedtls_aes_encrypt_ctr()` — sin instalar librerías externas
21. Generar un nonce aleatorio con `esp_random()` para cada mensaje
22. Medir y comparar el tiempo de cifrado: XOR vs AES-128 en el ESP32
23. Demostrar qué pasa si se reutiliza el mismo nonce con dos mensajes distintos

Punto de partida — integración con el lab base

En el código del laboratorio, localiza la función `xorCifrar()` y reemplázala por la llamada a tu mecanismo. La estructura del paquete y la lógica de ESP-NOW NO cambian:

```
// — ANTES (laboratorio base) —
void enviarSeguro(const char *mensaje) {
    PaqueteSeguro pkt;
    pkt.longitud = strlen(mensaje);
    // TODO Grupo 4: reemplaza esta línea por tu mecanismo
    xorCifrar((uint8_t*)mensaje, pkt.payload, pkt.longitud, CLAVE, sizeof(CLAVE));
    esp_now_send(macReceptor, (uint8_t*)&pkt, sizeof(pkt));
}

// — DESPUÉS (tu implementación) —
void enviarSeguro(const char *mensaje) {
    PaqueteSeguro pkt;
    pkt.longitud = strlen(mensaje);
    tuMecanismo_cifrar((uint8_t*)mensaje, pkt.payload, pkt.longitud /*, params */);
    esp_now_send(macReceptor, (uint8_t*)&pkt, sizeof(pkt));
}
```

Preguntas que debes responder en la exposición

- ¿Cuántas rondas tiene AES-128 y qué operación realiza en cada ronda?
- ¿Por qué el modo ECB es inseguro para imágenes? (Mostrar el pingüino de Linux cifrado con ECB)

- ¿Qué ventaja tiene CTR sobre CBC para mensajes de ESP-NOW de largo variable?

Análisis comparativo obligatorio (incluir en slides)

Compara tu mecanismo con el XOR del laboratorio en estos 3 ejes:

1. Known-plaintext attack: ¿es vulnerable? ¿Por qué sí o no?
2. Reutilización de clave: ¿qué pasa si usas la misma clave dos veces?
3. Fuerza bruta: ¿cuántas claves posibles tiene tu mecanismo? ¿Cuánto tiempo tardaría un ataque?

Recursos sugeridos

- FIPS 197 — Advanced Encryption Standard (especificación NIST)
- Código del laboratorio base: función cifrarAES_CTR() — Parte 4 de la guía original
- AES Visualizer — University of Waterloo
- The ECB penguin: en.wikipedia.org/wiki/Block_cipher_mode_of_operation

Entregable específico de este grupo:

Código con mbedTLS AES-CTR + comparativa de tiempos XOR vs AES + demo del pingüino ECB

GRUPO 5 · ECDH + AES sobre ESP-NOW

Protocolo: ECDH (Curve25519) + AES — intercambio de claves sin canal seguro previo

El protocolo completo del laboratorio original — intercambio automático de claves

Objetivo

Implementar el sistema completo del laboratorio original de 2 clases pero condensado: ECDH para intercambiar una clave de sesión automáticamente, y AES-128 para cifrar los mensajes. Sin necesidad de acordar la clave de antemano.

Conceptos clave a dominar

- Problema del laboratorio base: la clave XOR hay que compartirla de forma segura previamente
- ECDH: dos partes derivan la misma clave secreta sin que viaje por el canal
- Curve25519: curva elíptica diseñada para ser rápida y segura en microcontroladores
- Handshake de 4 pasos: HELLO → PUB_KEY_A → PUB_KEY_B → DATA cifrado
- Perfect Forward Secrecy: si capturan la clave de hoy, no pueden descifrar el pasado

Tareas de implementación

24. Implementar el handshake ECDH completo usando mbedtls/ecdh.h en el ESP32
25. Verificar que el secreto compartido es idéntico en ambas placas (Monitor Serial)
26. Usar los primeros 16 bytes del secreto como clave AES-128-CTR para los mensajes
27. Medir el tiempo total del handshake ECDH — ¿cuántos ms tarda en el ESP32?

Punto de partida — integración con el lab base

En el código del laboratorio, localiza la función `xorCifrar()` y reemplázala por la llamada a tu mecanismo. La estructura del paquete y la lógica de ESP-NOW NO cambian:

```
// — ANTES (laboratorio base) —
void enviarSeguro(const char *mensaje) {
    PaqueteSeguro pkt;
    pkt.longitud = strlen(mensaje);
    // TODO Grupo 5: reemplaza esta línea por tu mecanismo
    xorCifrar((uint8_t*)mensaje, pkt.payload, pkt.longitud, CLAVE, sizeof(CLAVE));
    esp_now_send(macReceptor, (uint8_t*)&pkt, sizeof(pkt));
}

// — DESPUÉS (tu implementación) —
void enviarSeguro(const char *mensaje) {
    PaqueteSeguro pkt;
    pkt.longitud = strlen(mensaje);
    tuMecanismo_cifrar((uint8_t*)mensaje, pkt.payload, pkt.longitud /*, params */);
    esp_now_send(macReceptor, (uint8_t*)&pkt, sizeof(pkt));
}
```

Preguntas que debes responder en la exposición

- ¿Cómo es posible que dos partes lleguen al mismo secreto sin enviárselo nunca?
- ¿Qué pasa si un atacante captura pubA y pubB? ¿Puede calcular el secreto?

- ¿Qué es el problema del logaritmo discreto en curvas elípticas y por qué es difícil?



Análisis comparativo obligatorio (incluir en slides)

Compara tu mecanismo con el XOR del laboratorio en estos 3 ejes:

1. Known-plaintext attack: ¿es vulnerable? ¿Por qué sí o no?
2. Reutilización de clave: ¿qué pasa si usas la misma clave dos veces?
3. Fuerza bruta: ¿cuántas claves posibles tiene tu mecanismo? ¿Cuánto tiempo tardaría un ataque?

Recursos sugeridos

- RFC 7748 — Elliptic Curves for Security (Curve25519)
- Código completo: guía original del laboratorio ESP32 (Partes 2, 3 y 4)
- Visualizador de curvas elípticas: desmos.com/calculator
- mbedTLS documentation: [mbedtlsls/ecdh.h](https://mbedtls.com/ecdh.h)

Entregable específico de este grupo:

Sistema completo ECDH + AES funcionando + diagrama del handshake + medición de tiempos

GRUPO 6 · RSA sobre ESP-NOW

Protocolo: RSA-2048 — criptografía asimétrica por factorización

El algoritmo que inventó la criptografía de clave pública — y por qué no cifra mensajes largos

Objetivo

Implementar RSA en el ESP32 para cifrar pequeños bloques de datos. Descubrir por qué RSA NO se usa directamente para cifrar mensajes, sino solo para intercambiar claves, y cómo los sistemas reales combinan RSA + AES.

Conceptos clave a dominar

- RSA: cifrado asimétrico basado en la dificultad de factorizar números primos grandes
- Clave pública (e, n): cualquiera puede cifrar. Clave privada (d, n): solo el dueño descifra
- Limitación crítica: RSA-2048 solo puede cifrar 245 bytes por operación
- Por eso TLS y PGP usan RSA solo para cifrar la clave AES (cifrado híbrido)
- Rendimiento en ESP32: RSA-2048 tarda ~800 ms — AES-128 tarda < 1 ms

Tareas de implementación

28. Generar un par de claves RSA-2048 usando mbedtls/rsa.h en el ESP32
29. Cifrar un mensaje corto (< 200 bytes) con la clave pública del receptor
30. Medir el tiempo de generación de claves y de cifrado/descifrado en el ESP32
31. Implementar el esquema híbrido: RSA cifra la clave AES, AES cifra el mensaje real

Punto de partida — integración con el lab base

En el código del laboratorio, localiza la función `xorCifrar()` y reemplázala por la llamada a tu mecanismo. La estructura del paquete y la lógica de ESP-NOW NO cambian:

```
// — ANTES (laboratorio base) —
void enviarSeguro(const char *mensaje) {
    PaqueteSeguro pkt;
    pkt.longitud = strlen(mensaje);
    // TODO Grupo 6: reemplaza esta línea por tu mecanismo
    xorCifrar((uint8_t*)mensaje, pkt.payload, pkt.longitud, CLAVE, sizeof(CLAVE));
    esp_now_send(macReceptor, (uint8_t*)&pkt, sizeof(pkt));
}

// — DESPUÉS (tu implementación) —
void enviarSeguro(const char *mensaje) {
    PaqueteSeguro pkt;
    pkt.longitud = strlen(mensaje);
    tuMecanismo_cifrar((uint8_t*)mensaje, pkt.payload, pkt.longitud /*, params */);
    esp_now_send(macReceptor, (uint8_t*)&pkt, sizeof(pkt));
}
```

Preguntas que debes responder en la exposición

- Si RSA-2048 tarda 800 ms por operación, ¿cuántos mensajes por segundo puede manejar?
- ¿Por qué la computación cuántica (algoritmo de Shor) amenaza RSA pero no AES?
- ¿Cómo implementa HTTPS el esquema híbrido RSA + AES en la práctica?

Análisis comparativo obligatorio (incluir en slides)

Compara tu mecanismo con el XOR del laboratorio en estos 3 ejes:

1. Known-plaintext attack: ¿es vulnerable? ¿Por qué sí o no?
2. Reutilización de clave: ¿qué pasa si usas la misma clave dos veces?
3. Fuerza bruta: ¿cuántas claves posibles tiene tu mecanismo? ¿Cuánto tiempo tardaría un ataque?

Recursos sugeridos

- RFC 8017 — PKCS #1: RSA Cryptography Specifications
- "A Method for Obtaining Digital Signatures" — Rivest, Shamir, Adleman (1978)
- mbedTLS: [mbedtls/rsa.h](https://mbedtls.com/rsa.h) — documentación y ejemplos
- Calculadora RSA online: números pequeños para entender el mecanismo

Entregable específico de este grupo:

Código con RSA-2048 + esquema híbrido RSA+AES + comparativa de tiempos + análisis de viabilidad en IoT

GRUPO 7 · HMAC-SHA256 sobre ESP-NOW

Protocolo: HMAC-SHA256 — autenticación e integridad de mensajes

Garantizar que el mensaje no fue alterado — aunque no esté cifrado

Objetivo

Implementar HMAC-SHA256 para añadir autenticación e integridad a los mensajes ESP-NOW. A diferencia del cifrado, el objetivo aquí NO es confidencialidad sino verificar que el mensaje es auténtico y no fue modificado en tránsito.

Conceptos clave a dominar

- Hash criptográfico SHA-256: cualquier cambio en el mensaje cambia completamente el hash
- HMAC: hash con clave secreta — garantiza que solo quien tiene la clave pudo generarlo
- $HMAC(K, M) = H((K \oplus opad) || H((K \oplus ipad) || M))$
- Diferencia entre cifrado (confidencialidad) y MAC (autenticidad)
- Ataque de extensión de longitud: por qué $H(K || M)$ no es seguro y HMAC sí lo es

Tareas de implementación

32. Calcular HMAC-SHA256 de cada mensaje con `mbedtls/md.h` y añadirlo al paquete ESP-NOW
33. En el receptor, recalculan el HMAC y comparar — rechazar si no coincide
34. Simular un ataque: modificar un byte del mensaje en tránsito y ver que el receptor lo detecta
35. Combinar: HMAC para integridad + XOR para confidencialidad (cifrado autenticado básico)

Punto de partida — integración con el lab base

En el código del laboratorio, localiza la función `xorCifrar()` y reemplázala por la llamada a tu mecanismo. La estructura del paquete y la lógica de ESP-NOW NO cambian:

```
// — ANTES (laboratorio base) —
void enviarSeguro(const char *mensaje) {
    PaqueteSeguro pkt;
    pkt.longitud = strlen(mensaje);
    // TODO Grupo 7: reemplaza esta línea por tu mecanismo
    xorCifrar((uint8_t*)mensaje, pkt.payload, pkt.longitud, CLAVE, sizeof(CLAVE));
    esp_now_send(macReceptor, (uint8_t*)&pkt, sizeof(pkt));
}

// — DESPUÉS (tu implementación) —
void enviarSeguro(const char *mensaje) {
    PaqueteSeguro pkt;
    pkt.longitud = strlen(mensaje);
    tuMecanismo_cifrar((uint8_t*)mensaje, pkt.payload, pkt.longitud /*, params */);
    esp_now_send(macReceptor, (uint8_t*)&pkt, sizeof(pkt));
}
```

Preguntas que debes responder en la exposición

- ¿Un atacante que conoce el mensaje y el HMAC puede deducir la clave? ¿Por qué no?
- ¿Qué diferencia hay entre un HMAC y una firma digital RSA? ¿Cuándo usar cada uno?

- ¿Por qué se necesita tanto confidencialidad como integridad? ¿Qué ataque es posible si solo hay cifrado sin MAC?

Análisis comparativo obligatorio (incluir en slides)

Compara tu mecanismo con el XOR del laboratorio en estos 3 ejes:

1. Known-plaintext attack: ¿es vulnerable? ¿Por qué sí o no?
2. Reutilización de clave: ¿qué pasa si usas la misma clave dos veces?
3. Fuerza bruta: ¿cuántas claves posibles tiene tu mecanismo? ¿Cuánto tiempo tardaría un ataque?

Recursos sugeridos

- RFC 2104 — HMAC: Keyed-Hashing for Message Authentication
- mbedTLS: `mbdts/md.h` — MBEDTLS_MD_SHA256
- "Length Extension Attack" — explicación y demo online
- NIST FIPS 180-4 — Secure Hash Standard

Entregable específico de este grupo:

Código con HMAC en cada paquete + demo de detección de manipulación + análisis de confidencialidad vs integridad

GRUPO 8 · ChaCha20 sobre ESP-NOW

Protocolo: ChaCha20 — cifrado de flujo moderno diseñado para hardware limitado

El sucesor de RC4 que usa TLS 1.3 cuando no hay aceleración AES por hardware

Objetivo

Implementar ChaCha20, un cifrado de flujo moderno diseñado por Daniel Bernstein (el mismo de Curve25519). Es más rápido que AES en dispositivos sin instrucciones AES-NI por hardware, y es la alternativa estándar en TLS 1.3.

Conceptos clave a dominar

- Cifrado de flujo: genera un keystream pseudoaleatorio y hace XOR con el mensaje
- ChaCha20: 20 rondas de operaciones ARX (suma, rotación, XOR) sobre 64 bytes de estado
- Clave de 256 bits + nonce de 96 bits + contador de 32 bits
- Por qué es resistente a ataques de timing (operaciones en tiempo constante)
- ChaCha20-Poly1305: cifrado autenticado (AEAD) — estándar en TLS 1.3 y WireGuard

Tareas de implementación

36. Implementar ChaCha20 usando mbedtls/chacha20.h disponible en mbedTLS del ESP32
37. Cifrar y descifrar mensajes ESP-NOW con clave de 32 bytes y nonce de 12 bytes
38. Medir y comparar velocidad: ChaCha20 vs AES-128-CTR en el ESP32 (sin hardware AES)
39. Implementar ChaCha20-Poly1305 para añadir autenticación al cifrado

Punto de partida — integración con el lab base

En el código del laboratorio, localiza la función `xorCifrar()` y reemplázala por la llamada a tu mecanismo. La estructura del paquete y la lógica de ESP-NOW NO cambian:

```
// — ANTES (laboratorio base) —
void enviarSeguro(const char *mensaje) {
    PaqueteSeguro pkt;
    pkt.longitud = strlen(mensaje);
    // TODO Grupo 8: reemplaza esta línea por tu mecanismo
    xorCifrar((uint8_t*)mensaje, pkt.payload, pkt.longitud, CLAVE, sizeof(CLAVE));
    esp_now_send(macReceptor, (uint8_t*)&pkt, sizeof(pkt));
}

// — DESPUÉS (tu implementación) —
void enviarSeguro(const char *mensaje) {
    PaqueteSeguro pkt;
    pkt.longitud = strlen(mensaje);
    tuMecanismo_cifrar((uint8_t*)mensaje, pkt.payload, pkt.longitud /*, params */);
    esp_now_send(macReceptor, (uint8_t*)&pkt, sizeof(pkt));
}
```

Preguntas que debes responder en la exposición

- ¿Por qué ChaCha20 es más rápido que AES en CPUs sin instrucciones AES-NI?
- ¿Cuándo elige TLS 1.3 AES-GCM y cuándo ChaCha20-Poly1305?
- Explica qué es AEAD (Authenticated Encryption with Associated Data) y por qué es importante

Análisis comparativo obligatorio (incluir en slides)

Compara tu mecanismo con el XOR del laboratorio en estos 3 ejes:

1. Known-plaintext attack: ¿es vulnerable? ¿Por qué sí o no?
2. Reutilización de clave: ¿qué pasa si usas la misma clave dos veces?
3. Fuerza bruta: ¿cuántas claves posibles tiene tu mecanismo? ¿Cuánto tiempo tardaría un ataque?

Recursos sugeridos

- RFC 8439 — ChaCha20 and Poly1305 for IETF Protocols
- "ChaCha, a variant of Salsa20" — Daniel J. Bernstein (2008)
- mbedTLS: `mbedtls/chacha20.h` y `mbedtls/chachapoly.h`
- WireGuard technical whitepaper — uso de ChaCha20-Poly1305

Entregable específico de este grupo:

Código ChaCha20-Poly1305 funcional + benchmark vs AES + análisis de cuándo usar cada uno

GRUPO 9 · TLS/DTLS sobre ESP-NOW simulado

Protocolo: DTLS (Datagram TLS) — TLS adaptado a protocolos sin conexión

El protocolo que llevaría a producción el sistema del laboratorio

Objetivo

Investigar e implementar un handshake simplificado inspirado en DTLS (TLS sobre UDP/datagramas), que es el protocolo que se usaría en un sistema IoT real. Integrar todos los conceptos anteriores: intercambio de claves, cifrado, autenticación e integridad en un protocolo estructurado.

Conceptos clave a dominar

- ¿Por qué TLS no funciona sobre ESP-NOW? TLS requiere TCP (entrega garantizada y ordenada)
- DTLS: TLS adaptado para UDP/datagramas — maneja reordenamiento y pérdida de paquetes
- Flight: secuencia de mensajes del handshake DTLS con retransmisión
- Epoch y sequence number: cómo DTLS evita ataques de replay
- Cipher suite: ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256 — leer e interpretar

Tareas de implementación

40. Diseñar e implementar un mini-protocolo de 4 mensajes inspirado en DTLS sobre ESP-NOW
41. Incluir: intercambio de claves (ECDH simplificado), cifrado AES o ChaCha20, y número de secuencia anti-replay
42. Simular pérdida de paquetes: ¿qué pasa si el HELLO no llega? Implementar retransmisión
43. Comparar el diseño propio con el handshake real de DTLS 1.3 (RFC 9147)

Punto de partida — integración con el lab base

En el código del laboratorio, localiza la función `xorCifrar()` y reemplázala por la llamada a tu mecanismo. La estructura del paquete y la lógica de ESP-NOW NO cambian:

```
// — ANTES (laboratorio base) —
void enviarSeguro(const char *mensaje) {
    PaqueteSeguro pkt;
    pkt.longitud = strlen(mensaje);
    // TODO Grupo 9: reemplaza esta línea por tu mecanismo
    xorCifrar((uint8_t*)mensaje, pkt.payload, pkt.longitud, CLAVE, sizeof(CLAVE));
    esp_now_send(macReceptor, (uint8_t*)&pkt, sizeof(pkt));
}

// — DESPUÉS (tu implementación) —
void enviarSeguro(const char *mensaje) {
    PaqueteSeguro pkt;
    pkt.longitud = strlen(mensaje);
    tuMecanismo_cifrar((uint8_t*)mensaje, pkt.payload, pkt.longitud /*, params */);
    esp_now_send(macReceptor, (uint8_t*)&pkt, sizeof(pkt));
}
```

Preguntas que debes responder en la exposición

- ¿Qué problemas específicos de UDP/ESP-NOW tuvo que resolver DTLS que TLS no necesita?
- ¿Cómo previene DTLS los ataques de replay con datagramas desordenados?
- Si tuvieras que asegurar una flota de 1000 sensores ESP32, ¿usarías DTLS o ESP-NOW manual? ¿Por qué?

**Análisis comparativo obligatorio (incluir en slides)**

Compara tu mecanismo con el XOR del laboratorio en estos 3 ejes:

1. Known-plaintext attack: ¿es vulnerable? ¿Por qué sí o no?
2. Reutilización de clave: ¿qué pasa si usas la misma clave dos veces?
3. Fuerza bruta: ¿cuántas claves posibles tiene tu mecanismo? ¿Cuánto tiempo tardaría un ataque?

Recursos sugeridos

- RFC 9147 — The Datagram Transport Layer Security (DTLS) Protocol Version 1.3
- RFC 6347 — DTLS 1.2 (versión más implementada actualmente)
- wolfSSL para ESP32: biblioteca TLS/DTLS para microcontroladores
- "DTLS for the IoT" — paper IEEE sobre uso en dispositivos embebidos

Entregable específico de este grupo:

Mini-protocolo DTLS implementado + análisis vs DTLS real + reflexión sobre seguridad en producción IoT

Checklist de Entrega — Todos los Grupos

- ☐ Laboratorio base funcional: mensajes cifrados con XOR enviados y recibidos correctamente
- ☐ Mecanismo asignado implementado: reemplaza xorCifrar() en el código del lab
- ☐ Demo en vivo preparada: funciona entre dos ESP32 físicos
- ☐ Análisis comparativo listo: tabla XOR vs tu mecanismo en los 3 ejes de seguridad
- ☐ Slides (PDF) entregados antes de la fecha indicada (máx. 8 slides)
- ☐ Preguntas de exposición respondidas en el informe

Criterio de nota máxima

Demo funcionando en vivo sin errores.

El grupo explica el mecanismo con un ejemplo numérico concreto (aunque sea simplificado).

Se presenta un caso histórico real relacionado con el protocolo.

El análisis comparativo con XOR es cuantitativo, no solo cualitativo.